

Curso Profesional de JasperReports 6.20.0 Community

Módulo 2 — Diseño básico de informes

TEORÍA

AUTOR: JAIME GALLO

JasperReports Library 6.20.0 Community · Jaspersoft Studio 6.20.0 Community Edition · Java 8 · Proyecto EditorialReports

Puntos incluidos

- 2.1 — Bandas
- 2.2 — Texto estático y campos de texto
- 2.3 — Campos
- 2.4 — Imágenes
- 2.5 — Formato y estilos
- 2.6 — Expresiones

Estado del proyecto al inicio del módulo

El Módulo 2 comienza **exactamente desde el estado final validado de M1/1.6**. Se conservan los proyectos `EditorialReports` y `EditorialReportsJava`, el JRXML básico y el ciclo Java compilación → llenado → exportación. A partir de esa base, el módulo introduce datos reales de catálogo, profundiza en bandas y texto, amplía el modelo con nuevos campos, incorpora imágenes y termina con estilos reutilizables y condicionales.

Baseline técnico. Esta edición permanece en JasperReports Library 6.20.0 Community, Jaspersoft Studio 6.20.0 Community Edition y Java 8 porque ésta es la baseline acordada para M1-M2. Las afirmaciones y fragmentos incompatibles con el XSD/API real de 6.20.0 se han corregido y los seis checkpoints se validan mediante ejecución end-to-end.

Correcciones técnicas consolidadas. Los estilos usan `isDefault` y el atributo `style` para referenciar el estilo padre; las fuentes del proyecto usan DejaVu Sans con `jasperreports-fonts:6.20.0`; el ajuste de texto validado usa `textAdjust="StretchHeight"`; el texto con marcado usa `markup="styled"`; y el total de páginas se obtiene con `PAGE_NUMBER` evaluado con `evaluationTime="Report"`; `PAGE_COUNT` cuenta registros procesados en la página y no representa el total de páginas.

Punto 2.1 — Bandas

Módulo: 2 — Diseño básico de informes (3 horas)

Proyecto: EditorialReports — sistema de informes empresariales para una editorial

Punto: 2.1 — Bandas

Objetivos de aprendizaje

- Profundizar en el modelo de bandas del motor y en la relación entre banda y momento de emisión.
- Añadir, eliminar y reordenar bandas en Jaspersoft Studio.
- Comprender el comportamiento de la banda Detail con una fuente de datos que devuelve registros.
- Utilizar la banda Last Page Footer como sustituta de Page Footer en la última página.
- Ajustar la propiedad splitType de las bandas para controlar los saltos de página.
- Documentar el comportamiento de las bandas del proyecto EditorialReports.

Parte teórica

Bloque 1 — Definición y clasificación de bandas

Una banda es una franja horizontal del informe con una altura definida y un momento de emisión propio. El motor de JasperReports decide cuándo emitir cada banda según su posición en el modelo del informe y según el estado del llenado. Esta decisión no la toma el diseñador escribiendo código, sino que está implícita en la propia estructura del JRXML. El diseñador coloca los elementos en la banda cuyo momento de emisión coincide con el comportamiento deseado y el motor se encarga de repetir, saltar o cerrar la banda según corresponda. La banda es, por tanto, la unidad estructural del informe y la base sobre la que se construyen todos los elementos.

```
<detail>
  <band height="20">
    <textField>
      <reportElement x="0" y="0" width="300" height="20"/>
      <textFieldExpression><![CDATA[{$F{titulo}}]></textFieldExpression>
    </textField>
  </band>
</detail>
```

Línea 1	<detail> → declara la banda de detalle. Se emite una vez por cada registro de la fuente de datos.
Línea 2	<band height="20"> → define la banda con 20 unidades de informe de altura. Cada registro producirá una fila de esa altura.
Línea 3-6	textField con la expresión \$F{titulo}. El campo se resuelve con el valor del registro actual en cada emisión.

Las secciones del informe pueden clasificarse por su comportamiento durante el llenado. `title` y `summary` se generan una sola vez por informe. `pageHeader` aparece al comienzo de cada página y `pageFooter` ocupa el pie normal de cada página. `background` es una sección especial que se renderiza en todas las páginas por debajo del resto del contenido. `columnHeader` y `columnFooter` se generan por

columna. `detail` se intenta generar una vez por cada registro. `lastPageFooter` es condicional: si existe, sustituye al `pageFooter` normal en la última ocurrencia del pie de página; no debe describirse como una banda que se emite una vez en todas las páginas.

CLASIFICACIÓN DE BANDAS POR MOMENTO DE EMISIÓN

Una vez por informe:

- `title`
- `summary`

Asociadas a la página:

- `pageHeader` → al comienzo de cada página
- `pageFooter` → pie normal de página
- `background` → en todas las páginas, detrás del contenido
- `lastPageFooter` → sustituye al `pageFooter` en su última ocurrencia

Una vez por columna:

- `columnHeader`
- `columnFooter`

Una vez por registro:

- `detail`

Qué representa el diagrama: los cuatro grupos de bandas según su momento de emisión. La clasificación es la que determina qué tipo de contenido se coloca en cada una.

Por qué es relevante: permite decidir con precisión dónde colocar cada elemento sin recurrir a la prueba y error.

No todas las secciones son obligatorias. Un informe puede contener solo algunas y el motor genera las que existan cuando corresponda. El modelo admite además cabeceras y pies de grupo y la sección `noData`, por lo que no existe una categoría universal llamada «informe completo de nueve bandas». En el estado de `EditorialReports` construido en este punto se trabajan explícitamente `background`, `title`, `pageHeader`, `columnHeader`, `detail`, `columnFooter`, `pageFooter`, `lastPageFooter` y `summary`. La selección depende del comportamiento que deba tener el documento.

Bloque 2 — Bandas de cabecera: Title, Page Header y Column Header

Las tres bandas de cabecera comparten la función de presentar información al lector antes del contenido principal, pero se diferencian en la frecuencia con que se emiten. La banda `title` se emite una sola vez, al comienzo del informe. La banda `pageHeader` se emite al inicio de cada página. La banda `columnHeader` se emite al inicio de cada columna. En un informe de una sola columna, `pageHeader` y `columnHeader` se emiten en el mismo momento, pero su contenido suele ser distinto: `pageHeader` contiene elementos de identificación del informe, mientras que `columnHeader` contiene los encabezados de los datos tabulares.

```
<pageHeader>
  <band height="25">
    <staticText>
      <reportElement x="0" y="5" width="555" height="15"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="9" isItalic="true"/>
      </textElement>
      <text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text>
    </staticText>
  </band>
</pageHeader>
```

Línea 1	<code><pageHeader></code> → declara la banda de cabecera de página. Se emite al inicio de cada página.
Línea 2	<code><band height="25"></code> → define la banda con 25 unidades de informe de altura.
Línea 3-9	<code>staticText</code> con el título abreviado del informe en cursiva y tamaño reducido.

La banda title puede contener elementos que no dependen de los datos y que solo aparecen una vez. El logotipo de la editorial, el título completo del informe, la fecha de emisión y el nombre del departamento son candidatos habituales. La banda pageHeader puede contener el mismo título abreviado, el nombre del usuario o un identificador del informe. La banda columnHeader contiene los rótulos de los campos que se imprimen en la banda detail. La separación entre las tres bandas permite que un mismo dato aparezca con distinta presentación según el momento en que se emite.

DIFERENCIAS ENTRE LAS TRES BANDAS DE CABECERA

title	→ una vez por informe contenido: logotipo, título completo, fecha ejemplo: "Catálogo Editorial - Informe Conceptual"
pageHeader	→ una vez por página contenido: título abreviado, usuario, fecha ejemplo: "Catálogo Editorial (cursiva)"
columnHeader	→ una vez por columna contenido: encabezados de campos ejemplo: "Título" "Precio"

Qué representa el diagrama: los tres tipos de cabecera con su frecuencia de emisión y ejemplos de contenido. La distinción permite decidir qué información colocar en cada una.

Por qué es relevante: evita duplicar contenido entre bandas y aprovecha la frecuencia de emisión para reforzar la estructura del documento.

Bloque 3 — La banda Detail y la repetición

La banda detail es la única banda que se emite una vez por cada registro de la fuente de datos. Su comportamiento depende del número de registros que la fuente devuelva. Si la fuente devuelve 100 registros, la banda se emite 100 veces. Si la fuente devuelve 0 registros, la banda no se emite. Este comportamiento es el que convierte al informe en un documento dinámico que se adapta al volumen de datos. La banda detail contiene los campos del informe y, opcionalmente, expresiones que dependen del registro actual.

```
<detail>
  <band height="20">
    <textField>
      <reportElement x="0" y="0" width="300" height="20"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[{$F{titulo}}]></textFieldExpression>
    </textField>
    <textField pattern="#,##0.00">
      <reportElement x="300" y="0" width="100" height="20"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="10"/>
      </textElement>
      <textFieldExpression><![CDATA[{$F{precio}}]></textFieldExpression>
    </textField>
  </band>
</detail>
```

```
</band>
</detail>
```

Línea 1	<detail> → declara la banda de detalle.
Línea 2	<band height="20"> → banda con 20 unidades de informe de altura. Cada registro producirá una fila.
Línea 3-10	primer textField con el campo titulo. Ancho 300, alineación vertical centrada, fuente DejaVu Sans 10.
Línea 11-18	segundo textField con el campo precio. Ancho 100, patrón ###0.00, alineación vertical centrada.

El motor intenta generar la banda `detail` para cada registro y crea nuevas páginas cuando el espacio disponible se agota. La altura declarada de `detail` multiplicada por el número de registros permite estimar el volumen vertical del contenido repetido, pero **no determina por sí sola** el número final de páginas: también consumen espacio las cabeceras, los pies, `title`, `summary`, los elementos que se estiran y las reglas de división de bandas. En una página A4 de 842 unidades con márgenes superior e inferior de 20 quedan 802 unidades antes de descontar esas bandas fijas y variables. Por eso cualquier cálculo previo debe tratarse como una estimación y confirmarse con el `JasperPrint` real.

REPETICIÓN DE LA BANDA DETAIL

Área disponible por página (A4 vertical con márgenes de 20 px):

$842 - 20 \text{ (topMargin)} - 20 \text{ (bottomMargin)} = 802 \text{ px}$

Restar las bandas fijas (`title`, `pageHeader`, `columnHeader`, `columnFooter`, `pageFooter`, `summary`) según corresponda.

Con banda `Detail` de 20 px y 100 registros:

$100 \times 20 = 2000 \text{ px de contenido}$

$2000 / \sim 700 \text{ px útiles por página} \approx 3 \text{ páginas}$

El motor calcula este reparto automáticamente.

Qué representa el diagrama: el cálculo aproximado del número de páginas en función de la altura de la banda `detail` y del número de registros. El reparto real lo realiza el motor y puede diferir ligeramente.

Por qué es relevante: permite estimar el volumen del informe antes de generarlo y ajustar la altura de la banda si es necesario.

La banda `detail` puede repetirse varias veces por página. Cuando el motor detecta que no hay espacio suficiente para emitir la banda completa, decide si dividirla o desplazarla a la página siguiente según el valor de la propiedad `splitType`. Esta propiedad se estudia con detalle en el bloque 5. En informes con muchas filas, el diseñador puede ajustar la altura de la banda `detail` para conseguir el equilibrio entre densidad de información y legibilidad. Una banda muy compacta produce informes densos; una banda alta produce informes más extensos pero más legibles.

Bloque 4 — Bandas de cierre: Column Footer, Page Footer, Last Page Footer y Summary

Las cuatro bandas de cierre se emiten al final de distintas secciones del informe. La banda `columnFooter` se emite al final de cada columna. La banda `pageFooter` se emite al final de cada página. La banda `lastPageFooter` sustituye a `pageFooter` en la última página cuando ambas están definidas. La banda `summary` se emite una sola vez al final del informe. Estas cuatro bandas permiten cerrar el

documento con distintos niveles de agregación: por columna, por página, en la última página y al final del informe.

```
<pageFooter>
  <band height="30">
    <staticText>
      <reportElement x="0" y="5" width="400" height="20"/>
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="9"/>
      </textElement>
      <text><![CDATA[EditorialReports - Documento generado con JasperReports 6.20.0]]></text>
    </staticText>
    <textField>
      <reportElement x="400" y="5" width="155" height="20"/>
      <textElement textAlignment="Right" verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="9"/>
      </textElement>
      <textFieldExpression><![CDATA["Página " + $V{PAGE_NUMBER}]]></textFieldExpression>
    </textField>
  </band>
</pageFooter>
```

Línea 1	<code><pageFooter></code> → declara la banda de pie de página. Se emite al final de cada página.
Línea 2	<code><band height="30"></code> → banda con 30 unidades de informe de altura.
Línea 3-9	primer <code>staticText</code> con el texto <code>EditorialReports - Documento generado con JasperReports 6.20.0</code> .
Línea 10-16	segundo <code>textField</code> con la expresión <code>"Página " + \$V{PAGE_NUMBER}</code> alineado a la derecha.

La banda `lastPageFooter` se comporta de forma condicional. Cuando existe, sustituye al `pageFooter` normal **en la última ocurrencia del pie de página**. En un informe sencillo, sin un `summary` que genere páginas adicionales, esto coincide con la última página visible. Sin embargo, si `summary` se imprime en una página propia o se desborda a varias páginas, la última ocurrencia del pie normal puede no coincidir con la última página física del documento. Esta precisión es importante para no confundir «último page footer» con «pie de la última página en cualquier circunstancia».

COMPORTAMIENTO DE `pageFooter` Y `lastPageFooter`

```
Caso 1: solo pageFooter
  Página 1 → pageFooter
  Página 2 → pageFooter
  Página 3 → pageFooter

Caso 2: solo lastPageFooter
  Página 1 → (nada)
  Página 2 → (nada)
  Página 3 → lastPageFooter

Caso 3: ambos definidos
  Página 1 → pageFooter
  Página 2 → pageFooter
  Página 3 → lastPageFooter
```

Qué representa el diagrama: los tres casos posibles de configuración de los pies de página. El caso 3 es el más habitual en informes con contenido variable.

Por qué es relevante: permite decidir cuándo usar `pageFooter`, cuándo `lastPageFooter` y cuándo ambos. La elección afecta a la apariencia de la última página.

La banda `summary` se genera una sola vez por informe y aparece al final del contenido principal, pero **no necesariamente es la última sección que se genera**: el `columnFooter` y/o el `pageFooter` de la página final pueden procesarse después. `summary` puede comenzar en una página nueva mediante la configuración del informe y también saltará a otra página si no cabe en el espacio restante. En ella se colocan valores del conjunto completo, como recuentos o sumas. El total de páginas de este curso se muestra con `PAGE_NUMBER` evaluado con `evaluationTime="Report"`.

Bloque 5 — Background, orden de emisión y saltos de página

La banda `background` se emite en cada página, detrás del contenido de las demás bandas. Se utiliza para marcas de agua, fondos de color o elementos decorativos que se repiten en todas las páginas. Su contenido no interfiere con el de las demás bandas porque se dibuja primero y el resto de bandas se superpone a él. La banda `background` tiene la particularidad de que no se ve afectada por los saltos de página: se emite completa en cada página. Su altura puede ser cero si no se utiliza, pero debe estar declarada si el informe la requiere.

```
<background>
  <band height="0"/>
</background>
```

Línea 1	<code><background></code> → declara la banda de fondo. Se emite en cada página.
Línea 2	<code><band height="0"/></code> → banda con altura cero. No emite contenido visible pero mantiene la estructura del informe.

La propiedad `splitType` controla **cuándo se permite dividir una banda** al agotarse el espacio disponible. En JasperReports 6.20.0 los valores son `Stretch`, `Prevent` e `Immediate`. `Stretch` evita que la división ocurra dentro de la altura declarada de la banda, pero permite partir la parte que se haya estirado por encima de esa altura. `Prevent` intenta impedir la primera división y desplazar la banda a la página o columna siguiente; si allí vuelve a no caber, el motor puede permitir la división para evitar un ciclo infinito. `Immediate` permite dividir la banda tan pronto como resulte necesario, después de que se haya impreso al menos un elemento. **Immediate no significa «forzar un salto de página antes de la banda».**

```
<band height="20" splitType="Prevent">
  <reportElement .../>
</band>
```

Línea 1	<code><band height="20" splitType="Prevent"></code> → el motor intenta evitar la primera división de la banda y desplazarla completa. Si el mismo contenido vuelve a no caber en el nuevo espacio, puede permitir la división para no quedar atrapado en un ciclo.
----------------	--

VALORES DE `splitType`

<code>Stretch</code>	→ No divide dentro de la altura declarada de la banda. Si el contenido estira la banda, la parte estirada sí puede dividirse.
<code>Prevent</code>	→ Intenta impedir la primera división y mover la banda completa. En un intento posterior puede dividir para evitar un ciclo infinito.

Immediate → Permite dividir tan pronto como sea necesario después de que se haya impreso al menos un elemento. No fuerza un salto previo.

Qué representa el diagrama: los tres valores de `splitType` y su comportamiento. El valor por defecto es `Stretch`.

Por qué es relevante: permite controlar el comportamiento de las bandas al final de cada página y evitar saltos inesperados o bandas cortadas.

Conviene distinguir **el orden estructural del JRXML** del **momento real de renderizado**. La plantilla declara sus secciones en el orden admitido por el esquema; durante el llenado, el motor genera cada sección cuando corresponde: `title` una vez al inicio, `pageHeader` por página, `columnHeader` por columna, `detail` por registro, pies cuando se cierra la columna o la página y `summary` una vez al final del contenido. `background` es especial: se dibuja en todas las páginas por debajo del resto. `lastPageFooter` solo sustituye al pie normal en su última ocurrencia. Los grupos y `noData`, si existen, añaden sus propias reglas. Por eso no debe memorizarse una lista lineal como si todas las secciones se emitiesen siempre una detrás de otra.

Resumen rápido de la teoría

- Una banda es una franja horizontal con una altura y un momento de emisión propios.
- Las secciones se distinguen por su momento de generación: una vez por informe, por página, por columna, por registro y casos especiales como `background` o `lastPageFooter`.
- Las bandas de cabecera son `title`, `pageHeader` y `columnHeader`.
- La banda `detail` se emite una vez por registro y es la que repite el contenido variable.
- Las bandas de cierre son `columnFooter`, `pageFooter`, `lastPageFooter` y `summary`.
- `lastPageFooter` sustituye al `pageFooter` normal en la última ocurrencia del pie de página; con `summary` multipágina esa ocurrencia no tiene por qué coincidir con la última página física.
- La banda `background` se emite en cada página detrás del contenido.
- La propiedad `splitType` controla el comportamiento de la banda al final de la página.

Punto 2.2 — Texto estático y campos de texto

Módulo: 2 — Diseño básico de informes (3 horas)

Proyecto: EditorialReports — sistema de informes empresariales para una editorial

Punto: 2.2 — Texto estático y campos de texto

Objetivos de aprendizaje

- Diferenciar las propiedades comunes y específicas de `staticText` y `textField`.
- Utilizar el atributo `markup` para interpretar etiquetas HTML en el contenido.
- Configurar `textAdjust="StretchHeight"` e `isBlankWhenNull` en campos de texto y reconocer `isStretchWithOverflow` como atributo heredado/deprecado en 6.20.0.
- Aplicar alineaciones, tipografías, colores y bordes a los elementos textuales.
- Combinar textos estáticos y campos de texto para construir encabezados y etiquetas.
- Documentar las propiedades de los elementos textuales en el proyecto EditorialReports.

Parte teórica

Bloque 1 — Propiedades comunes de los elementos textuales

Los elementos `staticText` y `textField` comparten un conjunto de propiedades que determinan su apariencia y su comportamiento en el documento. Estas propiedades se declaran en dos bloques anidados: `reportElement` y `textElement`. El bloque `reportElement` contiene las propiedades geométricas y de comportamiento: posición (`x`, `y`), tamaño (`width`, `height`), visibilidad (`isRemoveLineWhenBlank`), color de fondo (`backcolor`) y transparencia (`mode`). El bloque `textElement` contiene las propiedades tipográficas y de alineación: color del texto (`forecolor`), alineación horizontal (`textAlignment`), alineación vertical (`verticalAlignment`), rotación (`rotation`) y el elemento `font` con sus atributos. La separación entre ambos bloques refleja la diferencia entre dónde se coloca el elemento y cómo se presenta su contenido.

```
<staticText>
  <reportElement x="0" y="5" width="200" height="20" forecolor="#000000" backcolor="#FFFFFF" mode="Opaque"/>
  <textElement textAlignment="Center" verticalAlignment="Middle" rotation="None">
    <font fontName="DejaVu Sans" size="12" isBold="true" isItalic="false" pdfFontName="Helvetica-Bold"/>
  </textElement>
  <text><![CDATA[Rótulo centrado]]></text>
</staticText>
```

Línea 2

`<reportElement x="0" y="5" width="200" height="20" forecolor="#000000" backcolor="#FFFFFF" mode="Opaque"/>` → define la posición (0, 5), el tamaño (200 × 20), el color del texto (`forecolor`), el color de fondo (`backcolor`) y el modo de opacidad (`mode="Opaque"`). El modo `Opaque` hace que el fondo se rellene con `backcolor`; el modo `Transparent` lo deja transparente.

Línea 3

`<textElement textAlignment="Center" verticalAlignment="Middle" rotation="None">` → configura la alineación horizontal centrada, la alineación vertical centrada y la rotación nula. Los valores de `rotation` son `None`, `Left`, `Right` y `UpsideDown`.

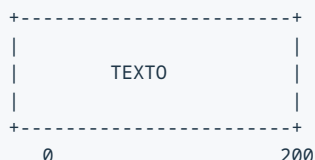
Línea 4

`<font fontName="DejaVu Sans" size="12" isBold="true" isItalic="false" pdfFontName="Helvetica-Bold"/>` → define la tipografía, el tamaño, la negrita y la cursiva. El atributo `pdfFontName` es específico para la exportación a PDF y permite mapear la fuente lógica a una fuente PDF estándar.

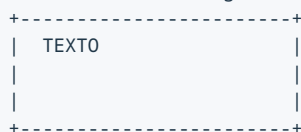
La alineación horizontal del texto dentro del cuadro se controla con el atributo `textAlignment`. Los valores son `Left`, `Center`, `Right` y `Justified`. La alineación vertical se controla con el atributo `verticalAlignment` y los valores son `Top`, `Middle` y `Bottom`. La combinación de ambos atributos determina la posición del texto dentro del rectángulo definido por `reportElement`. Si el texto es más ancho que el cuadro, se recorta por la derecha o se ajusta en varias líneas según la configuración `textAdjust`. Si el texto es más alto que el cuadro, se recorta por abajo o se expande la banda según la configuración de ajuste de texto. La coherencia entre el tamaño del cuadro y el contenido es responsabilidad del diseñador.

ALINEACIÓN Y POSICIÓN DEL TEXTO

`reportElement` define el rectángulo: `x=0, y=0, width=200, height=40`
`textElement.textAlignment = Center` → centra horizontalmente
`textElement.verticalAlignment = Middle` → centra verticalmente



Cambiar `verticalAlignment` a `Top`:



Qué representa el diagrama: la posición del texto dentro del rectángulo según los atributos de alineación. La alineación no mueve el rectángulo, solo el texto dentro de él.

Por qué es relevante: permite colocar el texto con precisión sin modificar la posición del cuadro, lo que simplifica el mantenimiento cuando se ajustan las dimensiones.

La propiedad `mode` controla la opacidad del fondo del elemento. El valor `Opaque` rellena el fondo con el color indicado en `backcolor`. El valor `Transparent` deja el fondo sin rellenar, de modo que se ve el color de la banda o de la página. La propiedad `isRemoveLineWhenBlank` controla si el elemento desaparece cuando su contenido está vacío. El valor `true` elimina el elemento y el espacio que ocupa; el valor `false` mantiene el espacio aunque el contenido esté vacío. Esta propiedad resulta útil en combinación con campos que pueden estar nulos, porque evita dejar líneas vacías en el documento.

Bloque 2 — Propiedades específicas del texto estático

El elemento `staticText` contiene un literal fijado en el diseño. Su estructura habitual incluye `reportElement`, `textElement` y `text`. Cuando el contenido necesita marcado en línea, JasperReports 6.20.0 utiliza el atributo `markup` del `textElement`. Para el marcado propio de JasperReports se usa `markup="styled"`.

```
<staticText>
  <reportElement x="0" y="5" width="300" height="20"/>
  <textElement markup="styled">
```

```

    <font fontName="DejaVu Sans" size="10"/>
  </textElement>
  <text><![CDATA[Este texto tiene <b>negrita</b> y <i> cursiva</i> en línea.]]></text>
</staticText>

```

Línea 2

reportElement define posición y tamaño.

Línea 3: markup="styled" indica que el contenido debe interpretarse con el marcado estilizado soportado por JasperReports.

Línea 6: el bloque CDATA permite escribir las etiquetas de marcado sin que el parser XML las trate como estructura del JRXML.

El material fuente usaba la denominación histórica markup="styled". En la plantilla validada del curso se emplea textElement markup="styled", que es la forma compatible con el XSD real de JasperReports 6.20.0. Si markup se omite, el contenido se trata como texto sin ese marcado.

```

<staticText>
  <reportElement x="0" y="5" width="300" height="20"/>
  <textElement markup="styled">
    <font fontName="DejaVu Sans" size="10"/>
  </textElement>
  <text><![CDATA[Precio: <b>19,95 €</b>]]></text>
</staticText>

```

Qué representa el ejemplo: un texto literal cuyo fragmento de precio se renderiza en negrita sin convertir el elemento en un campo dinámico.

Bloque 3 — Propiedades específicas del campo de texto

El elemento textField contiene una expresión evaluada durante el llenado. Además de reportElement, textElement y textFieldExpression, puede definir propiedades como pattern, isBlankWhenNull, evaluationTime y textAdjust.

En JasperReports 6.20.0 el XSD todavía admite isStretchWithOverflow, pero lo marca como **deprecated** y lo sustituye por textAdjust. Por eso el código validado del curso usa textAdjust="StretchHeight".

```

<textField textAdjust="StretchHeight" isBlankWhenNull="true" pattern="#,##0.00">
  <reportElement x="300" y="0" width="100" height="20"/>
  <textElement textAlignment="Right" verticalAlignment="Middle">
    <font fontName="DejaVu Sans" size="10"/>
  </textElement>
  <textFieldExpression><![CDATA[{$F{precio}]]></textFieldExpression>
</textField>

```

Línea 1

textAdjust="StretchHeight" permite aumentar la altura del campo para acomodar el contenido; isBlankWhenNull="true" evita imprimir null; pattern controla el formato.

Línea 2: reportElement define la caja inicial.

Línea 3: el texto se alinea a la derecha y se centra verticalmente.

Línea 6: la expresión toma el valor de {\$F{precio}} del registro actual.

COMPORTAMIENTO DE textAdjust

CutText (comportamiento normal)

+-----+

| Texto muy largo | <- se recorta si no cabe

```
+-----+
StretchHeight
+-----+
| Texto muy largo |
| que ocupa varias |
| líneas          |
+-----+
El campo aumenta de altura para mostrar el contenido.
```

Qué representa el diagrama: la diferencia entre mantener una altura fija y permitir que el campo crezca. En los ejercicios del módulo se utiliza `StretchHeight` para títulos que pueden ocupar más de una línea.

Por qué es relevante: evita enseñar como opción principal un atributo que la propia rama 6.20.0 ya declara deprecado, aunque siga siendo aceptado por compatibilidad.

Bloque 4 — Formato de contenido: patrones y estilos

El atributo `pattern` del `textField` aplica un formato al valor devuelto por la expresión. Los patrones siguen las reglas de `SimpleDateFormat` para fechas y de `DecimalFormat` para números. Un patrón de fecha como `dd/MM/yyyy` produce `22/09/2026`. Un patrón numérico como `#,##0.00` produce `1.234,56` en la configuración regional española. El patrón se aplica en el momento de la emisión y no modifica el valor almacenado en el objeto `JasperPrint`. El valor original sigue siendo el número o la fecha sin formato; el patrón solo afecta a la presentación.

```
<textField pattern="#,##0.00 €">
  <reportElement x="300" y="0" width="100" height="20"/>
  <textElement textAlignment="Right" verticalAlignment="Middle"/>
  <textFieldExpression><![CDATA[${F{precio}}]></textFieldExpression>
</textField>
```

Línea 1

`<textField pattern="#,##0.00 €">` → declara el campo con un patrón que incluye el símbolo del euro. El patrón `#,##0.00 €` produce valores como `19,95 €`.

Línea 4

`<textFieldExpression><![CDATA[${F{precio}}]></textFieldExpression>` → la expresión devuelve un valor numérico. El patrón se aplica al valor y produce la cadena formateada.

Además del patrón, el elemento `textElement` permite aplicar estilos tipográficos al contenido. Los atributos más utilizados son `forecolor` para el color del texto, `backcolor` para el color de fondo del texto, `markup` para interpretar etiquetas HTML o RTF en el contenido y `font` para la tipografía. El atributo `markup` admite los valores `none`, `styled`, `html` y `rtf`. El valor `none` imprime el contenido como texto literal. El valor `styled` interpreta las etiquetas simples de JasperReports. El valor `html` interpreta un subconjunto de etiquetas HTML. El valor `rtf` interpreta etiquetas RTF. La elección del valor depende del tipo de contenido que se quiera interpretar.

```
<textField>
  <reportElement x="0" y="0" width="300" height="20"/>
  <textElement markup="html">
    <font fontName="DejaVu Sans" size="10"/>
  </textElement>
  <textFieldExpression><![CDATA["<b>" + ${F{titulo}} + "</b>"]></textFieldExpression>
</textField>
```

Línea 3	<code><textElement markup="html"></code> → activa la interpretación de etiquetas HTML en el contenido del campo.
Línea 6	<code><textFieldExpression><![CDATA["" + \$F{titulo} + ""]]></textFieldExpression></code> → la expresión construye una cadena que envuelve el título con las etiquetas <code></code> y <code></code> . El motor interpreta las etiquetas y muestra el título en negrita.

Bloque 5 — Combinación de textos estáticos y campos de texto

La combinación de `staticText` y `textField` permite construir encabezados y etiquetas que mezclan contenido fijo y contenido variable. Un encabezado típico consta de un rótulo estático (Precio:) seguido de un campo de texto (`$F{precio}`). Los dos elementos se colocan en la misma banda y en posiciones contiguas. El rótulo identifica el dato y el campo lo muestra. Esta separación permite cambiar el formato del valor sin modificar el rótulo, y cambiar el texto del rótulo sin modificar la expresión. La combinación es la base de la mayoría de los encabezados de informes empresariales.

```
<staticText>
  <reportElement x="0" y="5" width="100" height="20"/>
  <textElement verticalAlignment="Middle">
    <font fontName="DejaVu Sans" size="10" isBold="true"/>
  </textElement>
  <text><![CDATA[Precio:]]></text>
</staticText>
<textField pattern="#,##0.00 €">
  <reportElement x="100" y="5" width="100" height="20"/>
  <textElement verticalAlignment="Middle">
    <font fontName="DejaVu Sans" size="10"/>
  </textElement>
  <textFieldExpression><![CDATA[$F{precio}]]></textFieldExpression>
</textField>
```

Línea 1-7	<code>staticText</code> con el rótulo Precio: en negrita. La posición <code>x="0"</code> y el ancho de 100 unidades de informe reservan el espacio para el rótulo.
Línea 8-14	<code>textField</code> con el valor del precio. La posición <code>x="100"</code> lo sitúa inmediatamente a la derecha del rótulo y el ancho de 100 unidades de informe reserva el espacio para el valor.

La combinación de elementos también permite construir encabezados con varios niveles. Un encabezado de sección puede contener un texto estático con el nombre de la sección, un campo con el número de registros y un campo con la suma de un valor. Cada elemento se coloca en una posición distinta dentro de la misma banda. El motor emite la banda una sola vez y los elementos se presentan en la posición definida. La alineación entre elementos se consigue ajustando las coordenadas y los anchos para que no haya solapamientos. La rejilla del editor facilita la alineación visual.

COMBINACIÓN DE RÓTULO Y CAMPO

x=0	x=100	x=200	x=300	x=400	
-----	-----	-----	-----		
Precio:	19,95 €				← rótulo + campo
-----	-----	-----	-----		
Total:	252,55 €				← rótulo + campo
-----	-----	-----	-----		

Qué representa el diagrama: la combinación de rótulos estáticos y campos de texto en una misma banda. Cada par ocupa dos posiciones contiguas y el conjunto se distribuye horizontalmente.

Por qué es relevante: permite construir encabezados y etiquetas que mezclan contenido fijo y variable con una distribución coherente.

La combinación de textos estáticos y campos de texto requiere atención a la alineación vertical. Si los dos elementos tienen alturas distintas, el texto puede quedar desalineado. La solución consiste en igualar las alturas y en configurar la alineación vertical de ambos al mismo valor (Middle, por ejemplo). La igualdad de alturas garantiza que los textos se alineen visualmente y que el conjunto resulte coherente. La propiedad `verticalAlignment` se declara en el bloque `textElement` de cada elemento y debe ser la misma en ambos. La coherencia entre elementos es una de las claves del diseño profesional de informes.

Resumen rápido de la teoría

- `staticText` y `textField` comparten las propiedades de `reportElement` y `textElement`.
- El bloque `reportElement` define posición, tamaño, colores y comportamiento.
- El bloque `textElement` define tipografía, alineación y rotación.
- `markup="styled"` permite interpretar etiquetas de estilo simples en el contenido.
- `textAdjust="StretchHeight"` es la opción validada para permitir que el campo aumente de altura con contenido largo. `isStretchWithOverflow` sigue existiendo en el XSD 6.20.0, pero está deprecado.
- `isBlankWhenNull` hace que el campo se muestre vacío cuando la expresión devuelve null.
- El atributo `pattern` aplica formato a fechas y números.
- El atributo `markup` permite interpretar HTML o RTF en el contenido.
- La combinación de rótulos y campos construye encabezados y etiquetas.

Punto 2.3 — Campos

Módulo: 2 — Diseño básico de informes (3 horas)

Proyecto: EditorialReports — sistema de informes empresariales para una editorial

Punto: 2.3 — Campos

Objetivos de aprendizaje

- Comprender el papel del campo en el modelo de datos de JasperReports.
- Declarar campos en el JRXML y asignarles el tipo Java correcto.
- Diferenciar campo, parámetro y variable y elegir el correcto según el caso.
- Resolver campos desde una fuente de datos personalizada y desde un JRBeanCollectionDataSource.
- Depurar errores de resolución de campos con las herramientas de diagnóstico del entorno.
- Ampliar el modelo de datos del proyecto EditorialReports con campos adicionales.

Parte teórica

Bloque 1 — El campo como unidad de dato

Un campo en JasperReports representa un valor que cambia con cada registro de la fuente de datos. El motor solicita el valor al origen de datos a través del método `getFieldValue(JRField)`, que recibe como argumento el nombre del campo. La declaración del campo en el JRXML no crea el dato: solo describe su nombre y su tipo para que el motor sepa cómo solicitarlo y cómo interpretarlo. El valor real procede de la fuente de datos y se resuelve en el momento de la emisión de la banda que contiene la expresión `$F{}`. Un campo, por tanto, es la interfaz entre la plantilla y la fuente de datos.

```
<field name="titulo" class="java.lang.String"/>
<field name="precio" class="java.lang.Double"/>
<field name="fechaPublicacion" class="java.util.Date"/>
```

Línea 1	<code><field name="titulo" class="java.lang.String"/></code> → declara un campo llamado titulo de tipo cadena. El nombre debe coincidir con el que la fuente de datos utiliza en <code>getFieldValue</code> .
Línea 2	<code><field name="precio" class="java.lang.Double"/></code> → declara un campo llamado precio de tipo numérico. El tipo <code>java.lang.Double</code> permite aplicar patrones numéricos.
Línea 3	<code><field name="fechaPublicacion" class="java.util.Date"/></code> → declara un campo llamado fechaPublicacion de tipo fecha. El tipo <code>java.util.Date</code> permite aplicar patrones de fecha.

El nombre del campo es sensible a mayúsculas y minúsculas. Un campo declarado como titulo y solicitado como Titulo no se resuelve y el motor lanza `JRException: Field not found: Titulo`. La convención en el proyecto EditorialReports es usar nombres en minúscula inicial y notación camelCase para nombres compuestos (fechaPublicacion, precioConIva, nombreAutor). Esta convención coincide con la de las propiedades de las clases Java que alimentan la fuente de datos y evita conversiones innecesarias. La coherencia entre el nombre del campo en el JRXML y el nombre solicitado en `getFieldValue` es condición necesaria para que la resolución funcione.

RESOLUCIÓN DE UN CAMPO

Plantilla JRXML:

```
<field name="titulo" class="java.lang.String"/>
...
<textFieldExpression><![CDATA[${F{titulo}}]></textFieldExpression>
```

Motor de llenado:

Por cada registro de la fuente de datos:

1. Solicita el valor del campo "titulo" a la fuente.
2. La fuente devuelve el valor.
3. El motor evalúa la expresión con ese valor.
4. El resultado se imprime en el documento.

Fuente de datos:

```
public Object getFieldValue(JRField campo) {
    if ("titulo".equals(campo.getName())) {
        return actual.getTitulo();
    }
    return null;
}
```

Qué representa el diagrama: el recorrido del valor de un campo desde la fuente de datos hasta el documento, pasando por la expresión y la evaluación del motor. La resolución se realiza una vez por cada emisión de la expresión.

Por qué es relevante: permite comprender por qué un campo mal declarado produce un error de resolución y por qué el nombre debe coincidir exactamente con el que la fuente de datos utiliza.

Bloque 2 — Declaración de campos y tipos Java

La declaración de un campo consta de dos atributos obligatorios: name y class. El atributo name identifica el campo dentro del informe y se utiliza en las expresiones `$F{}`. El atributo class indica el tipo Java del valor que la fuente de datos devuelve. Los tipos más habituales son `java.lang.String`, `java.lang.Integer`, `java.lang.Double`, `java.lang.Boolean`, `java.util.Date` y `java.math.BigDecimal`. El tipo declarado debe coincidir con el tipo del valor devuelto por la fuente de datos; una discrepancia provoca `ClassCastException` en el momento de la evaluación.

```
<field name="titulo" class="java.lang.String"/>
<field name="paginas" class="java.lang.Integer"/>
<field name="precio" class="java.lang.Double"/>
<field name="disponible" class="java.lang.Boolean"/>
<field name="fechaPublicacion" class="java.util.Date"/>
```

Línea 1	<code><field name="titulo" class="java.lang.String"/></code> → campo de tipo cadena. Adecuado para textos cortos o largos.
Línea 2	<code><field name="paginas" class="java.lang.Integer"/></code> → campo de tipo entero. Adecuado para contadores y números sin decimales.
Línea 3	<code><field name="precio" class="java.lang.Double"/></code> → campo de tipo doble. Adecuado para importes con decimales.
Línea 4	<code><field name="disponible" class="java.lang.Boolean"/></code> → campo de tipo booleano. Adecuado para indicadores de sí/no.
Línea 5	<code><field name="fechaPublicacion" class="java.util.Date"/></code> → campo de tipo fecha. Adecuado para fechas y horas.

La elección del tipo afecta al comportamiento del motor en dos aspectos. El primero es la conversión automática de tipos: si un campo declarado como `java.lang.String` devuelve un número, el motor lo convierte a cadena antes de imprimirlo. El segundo es la aplicación de patrones: los patrones numéricos solo se aplican a campos de tipo numérico y los patrones de fecha solo a campos de tipo fecha. Un patrón numérico aplicado a un campo de tipo cadena produce un error de formato. La declaración coherente del tipo evita conversiones implícitas y errores difíciles de diagnosticar.

TIPOS JAVA Y USOS RECOMENDADOS

Tipo	Uso recomendado
<code>java.lang.String</code>	Textos, códigos, nombres
<code>java.lang.Integer</code>	Contadores, cantidades sin decimales
<code>java.lang.Long</code>	Identificadores numéricos largos
<code>java.lang.Double</code>	Importes con decimales
<code>java.math.BigDecimal</code>	Importes de alta precisión
<code>java.lang.Boolean</code>	Indicadores sí/no
<code>java.util.Date</code>	Fechas y horas
<code>java.sql.Date</code>	Fechas de base de datos
<code>java.sql.Timestamp</code>	Marcas de tiempo de base de datos

Qué representa la tabla: los tipos Java más habituales en las declaraciones de campos y su uso recomendado. La elección del tipo debe corresponder al valor que la fuente de datos devuelve.

Por qué es relevante: permite declarar campos con el tipo correcto desde el primer momento y evitar conversiones implícitas que producen errores difíciles de localizar.

Bloque 3 — Campo, parámetro y variable

Los tres elementos comparten la sintaxis `$X{}` y se diferencian por la letra inicial: `$F{}` para campos, `$P{}` para parámetros y `$V{}` para variables. La diferencia no es solo sintáctica: cada uno tiene un ciclo de vida y un origen distintos. Un campo se resuelve una vez por cada registro de la fuente de datos y cambia con cada emisión. Un parámetro se resuelve una sola vez al inicio del llenado y permanece constante durante todo el proceso. Una variable se recalcula a lo largo del llenado según su expresión y su calculation. La elección del tipo correcto depende de la naturaleza del valor.

```
<parameter name="usuario" class="java.lang.String"/>
<field name="titulo" class="java.lang.String"/>
<variable name="ContadorLibros" class="java.lang.Integer" calculation="Count">
  <variableExpression><![CDATA[$F{titulo}]]></variableExpression>
</variable>
```

Línea 1	<code><parameter name="usuario" class="java.lang.String"/></code> → declara un parámetro de tipo cadena. Se resuelve una sola vez al inicio del llenado.
Línea 2	<code><field name="titulo" class="java.lang.String"/></code> → declara un campo de tipo cadena. Se resuelve una vez por cada registro de la fuente de datos.
Línea 3-5	<code><variable name="ContadorLibros" ...></code> → declara una variable de tipo entero que se calcula mediante Count sobre el campo titulo. Se recalcula a lo largo del llenado.

La diferencia entre los tres elementos determina qué tipo de valor puede representar cada uno. Un campo representa un valor que viene de la fuente de datos y que cambia con cada registro. Un parámetro representa un valor que se pasa desde el programa Java al inicio del llenado. Una variable representa un valor que el motor calcula a lo largo del llenado. Un contador de registros es una variable. Una suma de importes es una variable. Un nombre de usuario es un parámetro. Un título de

libro es un campo. La asignación correcta de cada valor al tipo de elemento adecuado es una de las decisiones de diseño que más afecta a la claridad de la plantilla.

DIFERENCIAS ENTRE CAMPO, PARÁMETRO Y VARIABLE

Aspecto	Campo	Parámetro	Variable
Sintaxis	\$F{}	\$P{}	\$V{}
Origen	Fuente datos	Programa	Cálculo
Resolución	Por registro	Al inicio	A lo largo
Cambia con registro	Sí	No	Sí (acumula)
Ejemplo	\$F{titulo}	\$P{usuario}	\$V{REPORT_COUNT}

Qué representa la tabla: las diferencias entre los tres tipos de elementos. La columna de origen indica de dónde procede el valor, y la columna de resolución indica cuándo se calcula.

Por qué es relevante: permite elegir el tipo correcto para cada valor y evitar errores de resolución. Un valor que viene de la fuente de datos siempre es un campo; un valor que se calcula a lo largo del llenado siempre es una variable.

Bloque 4 — Campos y fuentes de datos personalizadas

Una fuente de datos personalizada es una clase que implementa la interfaz JRDataSource y resuelve los campos mediante el método getFieldValue(JRField). El método recibe un objeto JRField que contiene el nombre del campo solicitado y su tipo declarado en el JRXML. La implementación del método compara el nombre recibido con los nombres que la clase puede resolver y devuelve el valor correspondiente del registro actual. Si el nombre no coincide con ninguno de los soportados, el método devuelve null y el motor imprime un valor vacío o un error según la configuración del campo.

```
@Override
public Object getFieldValue(JRField campo) {
    Libro actual = libros.get(indice);
    if ("titulo".equals(campo.getName())) {
        return actual.getTitulo();
    } else if ("precio".equals(campo.getName())) {
        return actual.getPrecio();
    } else if ("paginas".equals(campo.getName())) {
        return actual.getPaginas();
    } else if ("fechaPublicacion".equals(campo.getName())) {
        return actual.getFechaPublicacion();
    }
    return null;
}
```

Línea 1	@Override → anotación que indica que el método sobrescribe un método de la interfaz.
Línea 2	public Object getFieldValue(JRField campo) { → declara el método que el motor invoca para obtener el valor de un campo. Devuelve un Object genérico que el motor convierte al tipo declarado en el JRXML.
Línea 3	Libro actual = libros.get(indice); → obtiene el objeto del registro actual.
Línea 4-5	if ("titulo".equals(campo.getName())) { return actual.getTitulo(); } → si el nombre del campo es titulo, devuelve el título del libro.
Línea 6-7	else if ("precio".equals(campo.getName())) { return actual.getPrecio(); } → si el nombre es precio, devuelve el precio.

Línea 8-9	<code>else if ("paginas".equals(campo.getName())) { return actual.getPaginas(); } → si el nombre es paginas, devuelve el número de páginas.</code>
Línea 10-11	<code>else if ("fechaPublicacion".equals(campo.getName())) { return actual.getFechaPublicacion(); } → si el nombre es fechaPublicacion, devuelve la fecha.</code>
Línea 12	<code>return null; → si el nombre no coincide con ninguno de los soportados, devuelve null. El motor imprime un valor vacío o un error según la configuración del campo.</code>

JasperReports incluye implementaciones de `JRDataSource` para los casos más habituales. `JRBeanCollectionDataSource` recorre una colección de objetos Java y resuelve los campos a través de sus métodos `getter`. El nombre del campo debe coincidir con el nombre de la propiedad del objeto (sin el prefijo `get` y con la primera letra en minúscula). `JRMapCollectionDataSource` recorre una colección de mapas y resuelve los campos a través de las claves del mapa. `JResultSetDataSource` recorre un `ResultSet` de JDBC y resuelve los campos a través de las columnas del resultado. Estas implementaciones evitan tener que escribir una clase `JRDataSource` personalizada.

```
List<Libro> libros = Libro.listaEjemplo();
JRBeanCollectionDataSource dataSource = new JRBeanCollectionDataSource(libros);
```

Línea 1	<code>List<Libro> libros = Libro.listaEjemplo(); → obtiene la lista de libros.</code>
Línea 2	<code>JRBeanCollectionDataSource dataSource = new JRBeanCollectionDataSource(libros); → construye una fuente de datos que recorre la lista y resuelve los campos titulo y precio a través de los métodos <code>getTitulo()</code> y <code>getPrecio()</code> de la clase <code>Libro</code>.</code>

La resolución de campos por nombre a través de métodos `getter` sigue la convención `JavaBeans`. Un campo declarado como `titulo` en el `JRXML` se resuelve buscando el método `getTitulo()` en el objeto. Un campo declarado como `precio` se resuelve buscando `getPrecio()`. Un campo declarado como `fechaPublicacion` se resuelve buscando `getFechaPublicacion()`. Si el método no existe o el nombre no coincide, la resolución falla y el motor lanza `JRException: Field not found` o imprime un valor vacío según la configuración del campo. La convención `JavaBeans` es la que permite que `JRBeanCollectionDataSource` funcione sin necesidad de escribir código adicional.

Bloque 5 — Depuración de errores de resolución de campos

Los errores de resolución de campos son los más habituales cuando se trabaja con una fuente de datos personalizada. El motor los detecta en el momento de la evaluación y los reporta con un mensaje que incluye el nombre del campo. Los errores más frecuentes son tres. El primero es el error de nombre: el campo declarado en el `JRXML` no coincide con el nombre que la fuente de datos resuelve. El segundo es el error de tipo: el tipo declarado en el `JRXML` no coincide con el tipo del valor devuelto. El tercero es el error de declaración: el campo no está declarado en el `JRXML` pero se utiliza en una expresión `$F{}`.

ERRORES DE RESOLUCIÓN DE CAMPOS

```
Error 1: nombre incorrecto
JRXML:    <field name="titulo" .../>
Expresión: $F{Titulo}
Error:    Field not found: Titulo

Error 2: tipo incorrecto
JRXML:    <field name="precio" class="java.lang.Integer"/>
Fuente:    return new Double(19.95);
Error:    ClassCastException: java.lang.Double cannot be cast to java.lang.Integer
```

```
Error 3: campo no declarado
JRXML:      (sin declaración)
Expresión:  ${paginas}
Error:      Field not found: paginas
```

Qué representa el diagrama: los tres errores más frecuentes de resolución de campos, con el ejemplo mínimo de cada uno y el mensaje que produce el motor.

Por qué es relevante: permite diagnosticar con precisión el error y aplicar la corrección adecuada. Los tres errores se detectan en momentos distintos del ciclo de compilación y llenado.

La depuración de estos errores se realiza con tres herramientas. La primera es el panel Problems del entorno, que muestra los errores de compilación con el número de línea y la descripción. La segunda es la vista Console del programa Java, que muestra la traza completa de la excepción con el método y la línea donde se produjo. La tercera es la previsualización con un origen de datos vacío, que revela si el problema está en la declaración del campo o en la fuente de datos. La combinación de las tres herramientas permite localizar el error en pocos minutos.

HERRAMIENTAS DE DIAGNÓSTICO

Panel Problems	→ Errores de compilación del JRXML
Vista Console	→ Traza completa de la excepción
Vista Preview	→ Comportamiento con datos reales
Pestaña Source	→ Inspección del XML generado
Panel Outline	→ Verificación de la jerarquía del informe

Qué representa el diagrama: las cinco herramientas de diagnóstico disponibles en el entorno y su utilidad para depurar errores de resolución de campos.

Por qué es relevante: permite elegir la herramienta adecuada según el tipo de error. Los errores de sintaxis se detectan en Problems. Los errores de llenado se detectan en Console. Los errores de maquetación se detectan en Preview.

La mejor práctica para evitar errores de resolución de campos consiste en mantener una nomenclatura coherente entre el JRXML y las clases Java que alimentan la fuente de datos. Los nombres de los campos deben ser idénticos en ambos lados y deben seguir la convención JavaBeans. Los tipos deben coincidir exactamente o ser compatibles mediante conversión automática. La declaración de los campos debe realizarse antes de escribir las expresiones que los utilizan, para que el compilador pueda verificar su existencia. Estas tres prácticas reducen drásticamente el número de errores de resolución.

Resumen rápido de la teoría

- Un campo representa un valor que cambia con cada registro de la fuente de datos.
- La declaración de un campo consta de name y class.
- El nombre del campo es sensible a mayúsculas y debe coincidir con el de la fuente de datos.
- El tipo declarado debe coincidir con el tipo del valor devuelto.
- Los campos se referencian con `$F{}`, los parámetros con `$P{}` y las variables con `$V{}`.
- `JRBeanCollectionDataSource` resuelve campos por convención JavaBeans.
- Los errores más frecuentes son de nombre, de tipo y de declaración ausente.
- Las herramientas de diagnóstico son Problems, Console y Preview.

Punto 2.4 — Imágenes

Módulo: 2 — Diseño básico de informes (3 horas)

Proyecto: EditorialReports — sistema de informes empresariales para una editorial

Punto: 2.4 — Imágenes

Objetivos de aprendizaje

- Comprender el papel del elemento image en el modelo de elementos de JasperReports.
- Diferenciar las formas de cargar una imagen: ruta de archivo, classpath, URL y expresión dinámica.
- Aplicar los distintos modos de escala (Clip, FillFrame, RetainShape, RealHeight, RealSize).
- Configurar la propiedad onErrorType para controlar el comportamiento ante imágenes ausentes.
- Combinar imágenes estáticas y dinámicas en el informe del proyecto EditorialReports.
- Documentar el uso de imágenes en el proyecto.

Parte teórica

Bloque 1 — El elemento image

El elemento image de JasperReports permite insertar una imagen en cualquier banda del informe. La imagen se define mediante una expresión que el motor evalúa en el momento de la emisión y que puede devolver varios tipos de valor: una ruta de archivo como cadena, un objeto java.io.InputStream, un arreglo de bytes (byte[]), un objeto java.awt.Image o una URL. El elemento se compone de un bloque reportElement con las propiedades geométricas y un bloque imageExpression con la expresión que resuelve la imagen. Si se necesita transportar una imagen codificada en Base64, debe decodificarse desde la propia expresión a un tipo admitido (por ejemplo, byte[] o InputStream); no existe un segundo bloque image hijo destinado a incrustar Base64 dentro del elemento.

```
<image>
  <reportElement x="0" y="10" width="80" height="80" uuid="a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d"/>
  <imageExpression><![CDATA["resources/logo.png"]]></imageExpression>
</image>
```

Línea 1	<code><image></code> → declara un elemento de imagen. Puede colocarse en cualquier banda del informe.
Línea 2	<code><reportElement x="0" y="10" width="80" height="80" uuid="..." /></code> → define la posición (0, 10) y el tamaño (80 × 80) del elemento. La posición es relativa al borde superior izquierdo de la banda.
Línea 3	<code><imageExpression><![CDATA["resources/logo.png"]]></imageExpression></code> → expresión que devuelve la ruta del archivo de imagen. Las comillas dobles son obligatorias porque la expresión debe devolver una cadena. Sin las comillas, el motor interpreta la expresión como una variable o un campo y lanza un error de compilación.

La expresión de un elemento image puede devolver la imagen de cuatro formas distintas. La primera es una ruta de archivo como cadena. El motor abre el archivo en la ruta indicada y lo carga. La segunda

es un objeto `java.io.InputStream` que el motor lee y cierra. La tercera es un arreglo de bytes que el motor decodifica. La cuarta es un objeto `java.awt.Image` ya construido. La elección entre una forma y otra depende de dónde se encuentre la imagen y de cómo se obtenga. Para imágenes estáticas que residen en el classpath, la forma habitual es la ruta como cadena. Para imágenes almacenadas en la base de datos como BLOB, la forma habitual es el arreglo de bytes.

FORMAS DE RESOLVER UNA IMAGEN

Expresión	Tipo devuelto	Uso típico
"resources/logo.png"	<code>java.lang.String</code>	Archivo en disco
<code>\$P{rutaLogo}</code>	<code>java.lang.String</code>	Ruta parametrizada
<code>new java.io.FileInputStream(...)</code>	<code>java.io.InputStream</code>	Flujo de archivo
<code>\$F{imagenBlob}</code>	<code>byte[]</code>	Imagen en base de datos
<code>new javax.swing.ImageIcon(...)</code>	<code>java.awt.Image</code>	Imagen ya cargada
<code>new java.net.URL("http://...")</code>	<code>java.net.URL</code>	Imagen remota

Qué representa la tabla: las formas de resolver una imagen según el tipo devuelto por la expresión y el uso típico de cada una.

Por qué es relevante: permite elegir la forma correcta según el origen de la imagen y el tipo de dato disponible.

Bloque 2 — Carga desde archivo, classpath y URL

La carga de una imagen desde un archivo en disco se realiza con una expresión que devuelve la ruta como cadena. El motor abre el archivo con la ruta indicada y lo carga en memoria. La ruta puede ser absoluta o relativa al directorio de ejecución del programa. Las rutas relativas son más portables y se recomiendan cuando la imagen reside dentro del proyecto. La ruta `resources/logo.png` es relativa a la raíz del proyecto Java y es la que se utiliza en el proyecto `EditorialReports`.

```
<image>
  <reportElement x="0" y="10" width="80" height="80" uuid="..." />
  <imageExpression><![CDATA["resources/logo.png"]]></imageExpression>
</image>
```

Línea 3

`<imageExpression><![CDATA["resources/logo.png"]]></imageExpression>` → expresión que devuelve la ruta relativa del archivo. El motor busca el archivo en el directorio de ejecución del programa.

La carga desde el classpath se realiza con una expresión que utiliza el método `getResourceAsStream` de la clase `Class`. Este método devuelve un `InputStream` que el motor lee para construir la imagen. La carga desde el classpath es la más portable porque no depende del directorio de ejecución del programa: los archivos del classpath se incluyen en el JAR de la aplicación y se localizan automáticamente. El método se invoca sobre la clase actual o sobre cualquier clase del proyecto, y la ruta comienza por una barra inclinada para indicar la raíz del classpath.

```
<image>
  <reportElement x="0" y="10" width="80" height="80" uuid="..." />
  <imageExpression><![CDATA[getClass().getResourceAsStream("/resources/logo.png")]></imageExpression>
</image>
```

Línea 3

`<imageExpression><![CDATA[getClass().getResourceAsStream("/resources/logo.png")]]></imageExpression>` → expresión que invoca al método `getResourceAsStream` sobre la clase actual. La ruta comienza por una barra inclinada para indicar la raíz del classpath. El método devuelve un `InputStream` que el motor lee.

La carga desde una URL se realiza con una expresión que construye un objeto `java.net.URL` o que devuelve la URL como cadena. El motor abre la conexión y descarga la imagen. Este uso es menos frecuente porque introduce una dependencia de red y un tiempo de espera adicional. Resulta útil cuando el informe debe mostrar el logotipo alojado en un servidor corporativo y no se quiere distribuir el archivo con la aplicación. La expresión puede incluir la URL como cadena literal o como expresión que resuelve la URL a partir de un parámetro.

```
<image>
  <reportElement x="0" y="10" width="80" height="80" uuid="..." />
  <imageExpression><![CDATA["https://editorial.example.com/logo.png"]]></imageExpression>
</image>
```

Línea 3

`<imageExpression><![CDATA["https://editorial.example.com/logo.png"]></imageExpression>` → expresión que devuelve la URL de la imagen. El motor abre la conexión y descarga la imagen en el momento de la emisión.

Bloque 3 — Modos de escala

El modo de escala determina cómo se ajusta la imagen al rectángulo definido por `reportElement`. JasperReports 6.20.0 ofrece cinco modos: `Clip`, `FillFrame`, `RetainShape`, `RealHeight` y `RealSize`. El modo `Clip` recorta la imagen para que quepa en el rectángulo sin deformarla: la imagen se muestra con su tamaño natural y las partes que sobresalen se ocultan. El modo `FillFrame` estira la imagen para que ocupe todo el rectángulo, aunque se deforme. El modo `RetainShape` escala la imagen proporcionalmente para que quepa completa dentro del rectángulo sin deformarse, dejando espacio vacío alrededor. El modo `RealHeight` escala la imagen proporcionalmente según la altura del rectángulo. El modo `RealSize` mantiene el tamaño natural de la imagen sin importar las dimensiones del rectángulo.

```
<image scaleImage="RetainShape">
  <reportElement x="0" y="10" width="80" height="80" uuid="..." />
  <imageExpression><![CDATA["resources/logo.png"]]></imageExpression>
</image>
```

Línea 1

`<image scaleImage="RetainShape">` → declara la imagen con el modo de escala `RetainShape`. La imagen se escala proporcionalmente para caber completa dentro del rectángulo de 80 × 80 unidades de informe sin deformarse. El espacio sobrante queda vacío.

MODOS DE ESCALA

Rectángulo del `reportElement`: 80 × 80

Clip



FillFrame



RetainShape

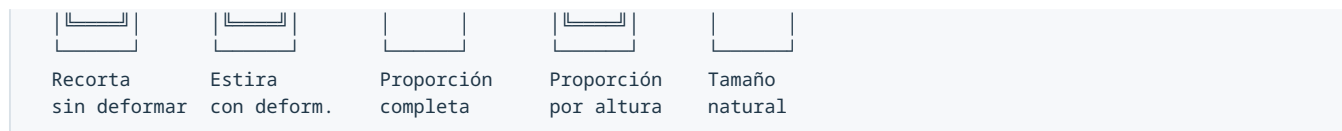


RealHeight



RealSize





Qué representa el diagrama: los cinco modos de escala aplicados a una misma imagen dentro de un rectángulo de 80 × 80. Cada modo produce un resultado visual distinto.

Por qué es relevante: permite elegir el modo adecuado según el uso de la imagen. Los logotipos suelen usar `RetainShape` para conservar la proporción. Los iconos suelen usar `RealSize` para mantener su tamaño natural.

Bloque 4 — Imágenes dinámicas y expresiones

Una imagen dinámica es una imagen cuya expresión depende del registro actual o de un parámetro. La expresión puede resolver el nombre del archivo a partir de un campo, construir la ruta con un parámetro o devolver un arreglo de bytes almacenado en la base de datos. Las imágenes dinámicas son útiles cuando el informe debe mostrar una imagen distinta para cada registro, como la portada de un libro o el logotipo del autor. La expresión se evalúa en el momento de la emisión de la banda que contiene el elemento y el motor carga la imagen correspondiente.

```
<image>
  <reportElement x="0" y="0" width="60" height="60" uuid="..." />
  <imageExpression><![CDATA["resources/portadas/" + $F{titulo} + ".png"]]></imageExpression>
</image>
```

Línea 3

`<imageExpression><![CDATA["resources/portadas/" + $F{titulo} + ".png"]]></imageExpression>` → expresión que construye la ruta de la imagen a partir del título del libro. Para cada registro, el motor busca el archivo correspondiente. Si el archivo no existe, la imagen no se muestra.

La propiedad `onErrorType` controla el comportamiento del elemento cuando la imagen no se puede cargar. Los valores posibles son `Error`, `Blank` e `Icon`. El valor `Error` lanza una excepción y detiene el llenado. El valor `Blank` deja el espacio vacío y continúa el llenado. El valor `Icon` muestra un icono genérico de error. El valor por defecto es `Error`. Para informes con imágenes dinámicas cuya existencia no está garantizada, el valor `Blank` resulta más robusto porque evita que el informe falle por una imagen ausente.

```
<image onErrorType="Blank">
  <reportElement x="0" y="0" width="60" height="60" uuid="..." />
  <imageExpression><![CDATA["resources/portadas/" + $F{titulo} + ".png"]]></imageExpression>
</image>
```

Línea 1

`<image onErrorType="Blank">` → declara la imagen con el modo de error `Blank`. Si la imagen no se puede cargar, el motor deja el espacio vacío y continúa el llenado sin lanzar excepción.

Bloque 5 — Combinación de imágenes y datos en el informe

Las imágenes se combinan con los demás elementos del informe para construir documentos visualmente ricos. El logotipo de la editorial se coloca habitualmente en la banda `title` o en la banda `pageHeader`. Las portadas de los libros se colocan en la banda `detail`. Los iconos de estado se colocan junto a los campos que representan. La combinación de imágenes y datos requiere atención a la

alineación y al tamaño, porque una imagen mal dimensionada puede descolocar toda la fila. La propiedad `hAlign` del elemento `image` permite alinear la imagen horizontalmente dentro de su rectángulo, con los mismos valores que la alineación del texto: `Left`, `Center` y `Right`.

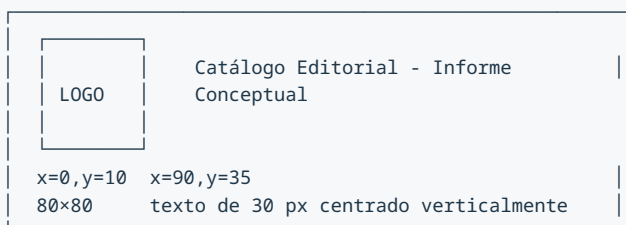
```
<title>
  <band height="100">
    <image hAlign="Left" vAlign="Middle">
      <reportElement x="0" y="10" width="80" height="80" uuid="..." />
      <imageExpression><![CDATA["resources/logo.png"]]></imageExpression>
    </image>
    <staticText>
      <reportElement x="90" y="40" width="465" height="30" uuid="..." />
      <textElement verticalAlignment="Middle">
        <font fontName="DejaVu Sans" size="18" isBold="true" />
      </textElement>
      <text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text>
    </staticText>
  </band>
</title>
```

Línea 3	<code><image hAlign="Left" vAlign="Middle"></code> → declara la imagen con alineación horizontal izquierda y vertical centrada dentro de su rectángulo.
Línea 4	<code><reportElement x="0" y="10" width="80" height="80" uuid="..." /></code> → posición y tamaño del elemento. El logotipo ocupa la esquina superior izquierda de la banda.
Línea 5	<code><imageExpression><![CDATA["resources/logo.png"]]></imageExpression></code> → expresión que devuelve la ruta del archivo del logotipo.
Línea 7-13	<code>staticText</code> con el título del informe. La posición <code>x="90"</code> lo sitúa a la derecha del logotipo.

La combinación de imágenes y datos requiere tener en cuenta el tamaño y la proporción. Una imagen de 80×80 unidades de informe en una banda de 100 unidades de informe de altura deja 10 unidades de informe de margen arriba y abajo. Un texto de 30 unidades de informe de altura centrado verticalmente en la misma banda queda alineado con el centro de la imagen si su coordenada Y es $(100 - 30) / 2 = 35$. La alineación precisa entre imagen y texto es una de las claves del diseño visual del informe. La rejilla del editor y las guías visuales facilitan esta alineación.

ALINEACIÓN DE IMAGEN Y TEXTO

Banda Title: altura 100 px



``Qué representa el diagrama: la alineación del logotipo y del título en la banda Title. La imagen ocupa la esquina izquierda y el texto se coloca a su derecha, centrado verticalmente.

****Por qué es relevante:**** permite construir encabezados visualmente equilibrados con imágenes y textos alineados.

Resumen rápido de la teoría

- El elemento `image` permite insertar imágenes en cualquier banda del informe.
- La expresión `imageExpression` puede devolver una ruta, un `InputStream`, un `byte[]` o un `java.awt.Image`.

- Las imágenes se cargan desde archivo, classpath o URL.
- Los modos de escala son Clip, FillFrame, RetainShape, RealHeight y RealSize.
- La propiedad onErrorType controla el comportamiento ante imágenes ausentes.
- Las imágenes dinámicas se resuelven a partir de campos o parámetros.
- La alineación entre imagen y texto requiere ajustar las coordenadas Y.

Punto 2.5 – Formato y estilos

****Módulo:** 2 – Diseño básico de informes (3 horas)**
****Proyecto:** EditorialReports – sistema de informes empresariales para una editorial**
****Punto:** 2.5 – Formato y estilos**

Objetivos de aprendizaje

- Comprender el papel del elemento `style` en la organización de la presentación del informe.
- Definir estilos reutilizables en el JRXML y aplicarlos a varios elementos.
- Diferenciar entre propiedades heredadas del estilo y propiedades sobrescritas en el elemento.
- Aplicar estilos condicionales mediante `conditionalStyle` y expresiones.
- Utilizar la herencia de estilos para construir jerarquías tipográficas coherentes.
- Documentar los estilos del proyecto EditorialReports.

Parte teórica

Bloque 1 – El elemento style

Un estilo en JasperReports es un conjunto de propiedades de presentación con un nombre que puede aplicarse a varios elementos. El estilo se declara en el JRXML con el elemento `style` y contiene un bloque `font`, un bloque `paragraph` y un bloque `box` que agrupan las propiedades tipográficas, de párrafo y de borde. Los elementos del informe referencian el estilo mediante el atributo `style` del bloque `reportElement`. Cuando un elemento referencia un estilo, hereda todas sus propiedades y puede sobrescribir las que necesite. Esta separación entre propiedades compartidas y propiedades específicas es la base de la organización de la presentación en un informe profesional.

```

<<xml
<style name="TituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#1A3D6B">
  <paragraph lineSpacing="Single"/>
</style>

```

Línea 1

`<style name="TituloPrincipal"` → declara un estilo llamado `TituloPrincipal`. El nombre es único dentro del informe y se utiliza para referenciarlo desde los elementos.

Línea 1 (continuación): `fontName="DejaVu Sans" fontSize="18" isBold="true" forecolor="#1A3D6B"` → define la tipografía, el tamaño, la negrita y el color del texto.

Línea 2: `<paragraph lineSpacing="Single"/>` → define el espaciado entre líneas. El valor `Single` indica espaciado simple.

Línea 3: `</style>` → cierra la declaración del estilo.

Los estilos se organizan en una jerarquía mediante el atributo `style`. Un estilo hijo hereda todas las propiedades de su padre y puede sobrescribir las que necesite. Esta herencia permite definir una tipografía base para el informe y derivar de ella variantes específicas con cambios mínimos. Un estilo `TituloPrincipal` puede heredar del estilo por defecto `Sans_Normal` y sobrescribir el tamaño y la negrita. Un estilo `TituloSecundario` puede heredar del mismo `Sans_Normal` y sobrescribir solo el tamaño. La herencia reduce la duplicación de propiedades y garantiza la coherencia tipográfica del documento.

```
<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>
<style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true"/>
<style name="TituloSecundario" style="Sans_Normal" fontSize="14" isBold="true"/>
<style name="TextoPequeno" style="Sans_Normal" fontSize="9"/>
```

Línea 1

`<style name="Sans_Normal" isDefault="true" fontName="DejaVu Sans" fontSize="10"/>` → declara el estilo por defecto del informe. Todos los elementos que no referencien otro estilo heredan estas propiedades.

Línea 2: `<style name="TituloPrincipal" style="Sans_Normal" fontSize="18" isBold="true"/>` → declara un estilo que hereda de `Sans_Normal` y sobrescribe el tamaño y la negrita. La tipografía sigue siendo DejaVu Sans.

Línea 3: `<style name="TituloSecundario" style="Sans_Normal" fontSize="14" isBold="true"/>` → declara un estilo similar al anterior pero con un tamaño menor.

Línea 4: `<style name="TextoPequeno" style="Sans_Normal" fontSize="9"/>` → declara un estilo para textos pequeños que hereda la tipografía pero reduce el tamaño.

Bloque 2 — Aplicación de estilos a elementos

Un elemento referencia un estilo mediante el atributo `style` del bloque `reportElement`. El motor aplica las propiedades del estilo al elemento y después aplica las propiedades declaradas directamente en el elemento, que sobrescriben las heredadas. Este orden de aplicación permite definir un estilo base y ajustar cada elemento según sus necesidades. El atributo `style` se aplica a `staticText`, `textField`, `image`, `line`, `rectangle`, `frame` y `subreport`. La aplicación del estilo no modifica la estructura del elemento: solo afecta a su presentación.

```
<staticText>
  <reportElement x="0" y="15" width="555" height="30" style="TituloPrincipal"/>
  <text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text>
</staticText>
```

Línea 1

`<staticText>` → declara el elemento de texto estático.

Línea 2: `<reportElement x="0" y="15" width="555" height="30" style="TituloPrincipal"/>` → define la posición y el tamaño del elemento y referencia el estilo `TituloPrincipal`. El elemento hereda la tipografía, el tamaño, la negrita y el color del estilo.

Línea 3: `<text><![CDATA[Catálogo Editorial - Informe Conceptual]]></text>` → contenido literal del texto.

La sobrescritura de propiedades se realiza declarando la propiedad directamente en el elemento. Si un elemento referencia el estilo `TituloPrincipal` pero necesita un color distinto, se declara el atributo `forecolor` en el bloque `textElement` del elemento. La propiedad declarada en el elemento tiene prioridad sobre la del estilo. Esta prioridad permite reutilizar estilos sin perder la capacidad de personalizar elementos concretos. La regla es que las propiedades específicas del elemento siempre sobrescriben las propiedades del estilo.

```
<staticText>
  <reportElement x="0" y="15" width="555" height="30" style="TituloPrincipal"/>
  <textElement forecolor="#FF0000"/>
  <text><![CDATA[Título en rojo]]></text>
</staticText>
```

Línea 2

`<reportElement x="0" y="15" width="555" height="30" style="TituloPrincipal"/>` → referencia el estilo `TituloPrincipal`. El elemento hereda la tipografía, el tamaño y la negrita del estilo.

Línea 3: `<textElement forecolor="#FF0000"/>` → sobrescribe el color del texto. El color rojo tiene prioridad sobre el color del estilo.

Bloque 3 — Estilos condicionales

Un estilo condicional es un conjunto de propiedades que se aplican a un elemento solo cuando se cumple una condición. Se declara dentro del propio elemento con el bloque `conditionalStyle` y contiene un bloque `conditionExpression` con la expresión que determina si el estilo se aplica y un bloque `style` con las propiedades. Los estilos condicionales permiten cambiar el formato de un elemento según el valor de un campo o de una variable. Un ejemplo típico es resaltar en rojo los precios superiores a un umbral o mostrar en negrita los libros disponibles.

```
<style name="TextoPrecio" style="Sans_Normal">
  <conditionalStyle>
    <conditionExpression><![CDATA[{precio} > 20]]></conditionExpression>
    <style forecolor="#CC0000" isBold="true"/>
  </conditionalStyle>
</style>
```

Línea 1

`<style name="TextoPrecio" style="Sans_Normal">` → declara un estilo que hereda de `Sans_Normal`.

Línea 2: `<conditionalStyle>` → abre el bloque de estilo condicional.

Línea 3: `<conditionExpression><![CDATA[{precio} > 20]]></conditionExpression>` → expresión que evalúa si el precio es superior a 20. Si devuelve verdadero, se aplica el estilo condicional.

Línea 4: `<style forecolor="#CC0000" isBold="true"/>` → define las propiedades que se aplican cuando la condición es verdadera: color rojo y negrita.

Línea 5: `</conditionalStyle>` → cierra el bloque de estilo condicional.

Línea 6: `</style>` → cierra la declaración del estilo.

Un estilo puede contener varios bloques `conditionalStyle`. El motor los evalúa en orden y aplica el último cuya condición sea verdadera. Este comportamiento permite definir varios niveles de formato según el valor del campo. Un primer bloque puede aplicar un formato para valores superiores a 30, un segundo para valores superiores a 20 y un tercero para valores superiores a 10. El orden de los bloques determina qué formato prevalece cuando varias condiciones son verdaderas. La convención es declarar primero las condiciones más específicas y después las más generales.

```
<style name="TextoPrecio" style="Sans_Normal">
  <conditionalStyle>
    <conditionExpression><![CDATA[{precio} > 30]]></conditionExpression>
    <style forecolor="#990000" isBold="true" fontSize="12"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[{precio} > 20]]></conditionExpression>
    <style forecolor="#CC0000" isBold="true"/>
  </conditionalStyle>
  <conditionalStyle>
    <conditionExpression><![CDATA[{precio} > 10]]></conditionExpression>
    <style forecolor="#006600"/>
  </conditionalStyle>
</style>
```

```
</conditionalStyle>
</style>
```

Línea 2-5

primer estilo condicional. Se aplica cuando el precio es superior a 30. El color es rojo oscuro, el texto en negrita y el tamaño 12.

Línea 6-9: segundo estilo condicional. Se aplica cuando el precio es superior a 20. El color es rojo medio y el texto en negrita.

Línea 10-13: tercer estilo condicional. Se aplica cuando el precio es superior a 10. El color es verde.

Bloque 4 — Herencia y organización de estilos

La organización de los estilos en una jerarquía refleja la organización tipográfica del documento. Un estilo base define la tipografía y el tamaño por defecto. Los estilos derivados definen las variantes para títulos, subtítulos, textos de tabla y notas. Esta jerarquía permite que un cambio en el estilo base se propague a todos los estilos derivados y a todos los elementos que los referencian. La modificación de la tipografía del informe se realiza en un único punto y el resto del documento se actualiza automáticamente. Esta propiedad es la que hace que los estilos sean una herramienta de mantenimiento y no solo de presentación.

JERARQUÍA DE ESTILOS DEL INFORME

```
Sans_Normal (default)
├── TituloPrincipal (fontSize=18, bold)
├── TituloSecundario (fontSize=14, bold)
├── TextoTabla (fontSize=9)
├── TextoTablaCabecera (fontSize=9, bold)
├── TextoPequeno (fontSize=9, italic)
├── TextoPrecio (fontSize=9 + condicionales)
└── TextoNota (fontSize=8, italic, forecolor=#666666)
```

Qué representa el diagrama: la jerarquía de estilos del informe. Todos los estilos derivan del estilo por defecto `Sans_Normal` y sobrescriben las propiedades que necesitan.

Por qué es relevante: permite organizar la presentación del informe de forma coherente y facilitar los cambios globales. Modificar la tipografía del estilo base cambia la tipografía de todo el documento.

Los estilos del informe se declaran como elementos de nivel superior del JRXML, antes de los campos, variables y bandas. No se declaran dentro de la banda `background`. Para reutilizar estilos entre informes se emplean plantillas de estilo JRTX u otros mecanismos de estilos externos. En este curso, los estilos locales se mantienen al principio del `jasperReport`, después de las propiedades y antes de los campos.

```
<style name="EstiloGlobal" style="Sans_Normal" fontSize="10"/>
```

Línea 1

declara un estilo local de nivel superior que hereda del estilo base mediante el atributo `style`. En JasperReports 6.20.0 los estilos del informe no se insertan dentro de una banda como `background`; deben aparecer en la zona de estilos del `jasperReport` antes de los campos y las bandas.

Bloque 5 — Buenas prácticas en el uso de estilos

El uso de estilos en un informe profesional sigue tres buenas prácticas. La primera es declarar un estilo por defecto con `isDefault="true"` para que todos los elementos que no referencien otro estilo hereden una presentación coherente. La segunda es agrupar las propiedades comunes en un estilo base y declarar los estilos derivados solo con las propiedades que cambian. La tercera es usar estilos condicionales para los formatos que dependen de los datos, en lugar de duplicar el elemento con dos variantes. Estas tres prácticas reducen la duplicación, facilitan el mantenimiento y garantizan la coherencia visual del documento.

ANTES Y DESPUÉS DE APLICAR ESTILOS

ANTES (sin estilos):

```
<staticText>
  <reportElement .../>
  <textElement textAlignment="Center" verticalAlignment="Middle">
    <font fontName="DejaVu Sans" size="18" isBold="true"/>
  </textElement>
  <text>Catálogo Editorial</text>
</staticText>

<staticText>
  <reportElement .../>
  <textElement textAlignment="Center" verticalAlignment="Middle">
    <font fontName="DejaVu Sans" size="18" isBold="true"/>
  </textElement>
  <text>Informe Conceptual</text>
</staticText>
```

(Las propiedades tipográficas se repiten en cada elemento.)

DESPUÉS (con estilos):

```
<style name="TituloPrincipal" fontName="DejaVu Sans" fontSize="18" isBold="true"/>

<staticText>
  <reportElement ... style="TituloPrincipal"/>
  <text>Catálogo Editorial</text>
</staticText>

<staticText>
  <reportElement ... style="TituloPrincipal"/>
  <text>Informe Conceptual</text>
</staticText>
```

(Las propiedades tipográficas se declaran una sola vez en el estilo.)

Qué representa el diagrama: la diferencia entre declarar las propiedades tipográficas en cada elemento y declararlas en un estilo reutilizable. La segunda opción reduce la duplicación y facilita los cambios.

Por qué es relevante: permite valorar el impacto del uso de estilos en el mantenimiento del informe. Un cambio en la tipografía del informe requiere modificar un solo estilo en lugar de todos los elementos.

La combinación de estilos y propiedades específicas permite construir informes visualmente ricos sin duplicar código. Un elemento hereda las propiedades del estilo y sobrescribe solo las que necesita. Una cabecera de tabla puede heredar la tipografía del estilo base y sobrescribir la negrita y el color de fondo. Un precio puede heredar la tipografía del estilo base y sobrescribir el color según un estilo condicional. La combinación de herencia y sobrescritura es la que da flexibilidad al sistema de estilos sin perder la coherencia global.

```
<style name="TextoPrecio" style="Sans_Normal">
  <box>
    <pen lineWidth="0.5" lineColor="#CCCCCC"/>
    <topPen lineWidth="0.5" lineColor="#CCCCCC"/>
    <bottomPen lineWidth="0.5" lineColor="#CCCCCC"/>
    <leftPen lineWidth="0.5" lineColor="#CCCCCC"/>
    <rightPen lineWidth="0.5" lineColor="#CCCCCC"/>
  </box>
</style>
```

Línea 1

`<style name="TextoPrecio" style="Sans_Normal">` → declara un estilo que hereda de `Sans_Normal`.

Línea 2: `<box>` → abre el bloque de borde y relleno.

Línea 3-7: `<pen .../>`, `<topPen .../>`, `<bottomPen .../>`, `<leftPen .../>` y `<rightPen .../>` → definen el grosor y el color de los bordes del elemento. El borde se aplica a los cuatro lados del rectángulo del elemento.

Línea 8: `</box>` → cierra el bloque de borde.

Línea 9: `</style>` → cierra la declaración del estilo.

Resumen rápido de la teoría

- Un estilo es un conjunto de propiedades de presentación con un nombre que se aplica a varios elementos.
- Los estilos se declaran con el elemento `style` y contienen bloques `font`, `paragraph` y `box`.
- Los estilos heredan de otros estilos mediante el atributo `style`.
- El estilo por defecto se declara con `isDefault="true"`.
- Un elemento referencia un estilo mediante el atributo `style` del bloque `reportElement`.
- Las propiedades declaradas en el elemento sobrescriben las del estilo.
- Los estilos condicionales aplican propiedades según una expresión.
- La jerarquía de estilos refleja la organización tipográfica del documento.

Punto 2.6 — Expresiones

Módulo: 2 — Diseño básico de informes (3 horas)

Proyecto: EditorialReports — sistema de informes empresariales para una editorial

Punto: 2.6 — Expresiones

Objetivos de aprendizaje

- Escribir expresiones Java válidas dentro de las plantillas JRXML.
- Utilizar operadores aritméticos, lógicos y de comparación en expresiones de informe.
- Invocar métodos de `String`, `Date`, `Double` y otras clases Java cuando el cálculo lo requiera.
- Combinar campos (`$F{}`), parámetros (`$P{}`) y variables (`$V{}`) en una misma plantilla.
- Distinguir el momento de evaluación de campos, parámetros y variables incorporadas por JasperReports.
- Depurar expresiones mediante Problems, Preview, Console y la ejecución Java del proyecto.
- Documentar las expresiones del proyecto EditorialReports.

Parte teórica

Bloque 1 — Qué es una expresión y qué contexto puede utilizar

Una expresión de JasperReports es una expresión Java compilada como parte del informe. Puede aparecer en un `textFieldExpression`, en la expresión de una variable, en una condición de estilo, en un `printWhenExpression`, en una expresión de imagen y en muchos otros puntos del JRXML. La expresión devuelve un valor y ese valor debe ser compatible con el contexto que lo consume. Un `textField` puede mostrar un `String`, un número o una fecha; una condición debe devolver un `Boolean`; una variable declarada como `java.lang.Double` debe recibir un resultado compatible con `Double`.

```
<textFieldExpression><![CDATA[$F{precio}.doubleValue() * 1.21d]]></textFieldExpression>
```

Línea 1

`<textFieldExpression>` → abre la expresión de un campo de texto. Línea 1 (continuación): `$F{precio}.doubleValue() * 1.21d` → obtiene el precio del registro actual, lo convierte a `double` y lo multiplica por 1,21. Línea 1 (continuación): `</textFieldExpression>` → cierra la expresión.

Los tres prefijos que más se utilizan en este módulo son `$F{}`, `$P{}` y `$V{}`. Un campo (`$F{}`) procede del registro actual de la fuente de datos. Un parámetro (`$P{}`) procede normalmente de la aplicación que ejecuta el informe, aunque puede tener un valor por defecto. Una variable (`$V{}`) pertenece al estado interno del informe y puede ser una variable definida por el usuario o una variable incorporada por el motor.

CONTEXTO DE UNA EXPRESIÓN EN DETAIL

```
Registro actual (JRDataSource)
├─ $F{titulo}
├─ $F{precio}
├─ $F{paginas}
├─ $F{fechaPublicacion}
└─ $F{disponible}
```

```

Parámetros
└─ ${usuario}

Variables
├─ ${REPORT_COUNT}
├─ ${PAGE_NUMBER}
├─ ${PAGE_COUNT}
├─ ${TotalPrecios}
└─ ${PrecioConIVA}

```

Qué representa el diagrama: las tres fuentes de valores disponibles para las expresiones del informe.

Por qué es relevante: evita confundir datos del registro con valores enviados por la aplicación o con valores calculados por el motor.

No debe suponerse que dos elementos visuales se comunican por el simple hecho de estar uno antes que otro en una banda. JasperReports sigue su ciclo de llenado y cada variable tiene además propiedades de cálculo, incremento, reinicio y evaluación. La práctica correcta consiste en expresar las dependencias de forma explícita mediante campos, parámetros y variables, no mediante efectos laterales entre elementos.

Bloque 2 — Operadores aritméticos, lógicos, de comparación y ternarios

Las expresiones usan la sintaxis de Java. Los operadores aritméticos +, -, *, / y % permiten calcular valores. Los operadores <, >, <=, >=, == y != permiten comparar valores. Los operadores lógicos &&, || y ! combinan condiciones. El operador ternario condición ? valor1 : valor2 permite escoger un resultado sin escribir un bloque if.

```

<textFieldExpression><![CDATA[
    ${precio}.doubleValue() > 20.0d && ${disponible}.booleanValue()
    ? "Destacado"
    : "Normal"
]]></textFieldExpression>

```

Línea 1

<textFieldExpression><![CDATA[→ abre la expresión y el bloque CDATA. Línea 2: `${precio}.doubleValue() > 20.0d` → compara el precio con 20. Línea 2 (continuación): `&&` → exige que la condición del precio y la disponibilidad sean verdaderas. Línea 2 (continuación): `${disponible}.booleanValue()` → obtiene el valor booleano del campo. Línea 3-4: `? "Destacado" : "Normal"` → devuelve una de las dos cadenas.

Las expresiones de informes deben ser legibles y, en lo posible, sin efectos laterales. Java dispone de operadores de asignación e incremento, pero utilizarlos para modificar estado desde un elemento de informe dificulta el razonamiento sobre el llenado y no es el patrón que se enseña en este curso. Los cálculos acumulativos se representan con variables de JasperReports.

Bloque 3 — Métodos Java y tratamiento de valores nulos

Las expresiones pueden invocar métodos Java. En EditorialReports se utilizan `length()` y `substring()` sobre cadenas, `after()` sobre fechas y `doubleValue()` sobre números. La posibilidad de invocar métodos permite construir cálculos compactos, pero también introduce el riesgo de `NullPointerException`. Si una fuente puede devolver null, la expresión debe comprobarlo antes de invocar métodos.

```

<textFieldExpression><![CDATA[
    ${titulo} == null

```

```
? "Sin título"
: ($F{titulo}.length() > 30 ? "Título largo" : "Título corto")
]]></textFieldExpression>
```

Línea 1

`$F{titulo} == null` → comprueba primero si el valor existe. Línea 2: `? "Sin título"` → devuelve un texto seguro si el campo es nulo. Línea 3: `$F{titulo}.length() > 30` → solo invoca `length()` cuando el campo no es nulo.

En el checkpoint 2.6 la fuente de datos del curso garantiza títulos, precios, fechas y disponibilidad no nulos, pero se mantiene la comprobación explícita en la variable `PrecioConIVA` para demostrar el patrón defensivo.

Bloque 4 — Parámetros, variables y variables incorporadas

El parámetro `usuario` representa un dato que llega desde la aplicación. El JRXML declara un valor por defecto para que el informe pueda previsualizarse desde Studio, mientras que el programa Java lo suministra explícitamente mediante el mapa de parámetros. Esta doble vía permite utilizar el mismo diseño tanto en Preview como en ejecución programática.

```
<parameter name="usuario" class="java.lang.String">
  <defaultValueExpression><![CDATA["Ana Martínez"]]></defaultValueExpression>
</parameter>
```

Línea 1

`<parameter ...>` → declara un parámetro de tipo `String`. Línea 2: `<defaultValueExpression>` → define el valor usado cuando la aplicación no envía el parámetro. Línea 3: `</parameter>` → cierra la declaración.

La variable `PrecioConIVA` se recalcula para cada registro. No tiene `calculation="Sum"`; por tanto, actúa como una variable cuyo valor se obtiene de la expresión actual.

```
<variable name="PrecioConIVA" class="java.lang.Double">
  <variableExpression><![CDATA[
    $F{precio} == null ? null : Double.valueOf($F{precio}.doubleValue() * 1.21d)
  ]]></variableExpression>
</variable>
```

Línea 1

declara la variable de tipo `Double`. Línea 2-4: calcula el precio con IVA y preserva `null` si el precio es nulo.

Las variables incorporadas deben interpretarse correctamente. `REPORT_COUNT` contiene el número de registros procesados en el informe. `PAGE_NUMBER` representa el número de página durante el llenado y, cuando un campo se evalúa con `evaluationTime="Report"`, puede utilizarse para imprimir el total final de páginas. `PAGE_COUNT`, en cambio, contiene el número de registros procesados en la página actual y se reinicia al cambiar de página; **no es el total de páginas**.

VARIABLES INCORPORADAS UTILIZADAS EN ESTE MÓDULO

```
REPORT_COUNT → registros procesados en el informe
PAGE_NUMBER  → número de página; evaluado a Report permite obtener el total final
PAGE_COUNT   → registros procesados en la página actual
COLUMN_NUMBER → número de columna actual
```

Qué representa el diagrama: el significado operativo de las variables incorporadas que aparecen en el curso.

Por qué es relevante: usar PAGE_COUNT como si fuera el total de páginas produce resultados conceptualmente incorrectos aunque la plantilla compile.

Bloque 5 — Depuración y validación de expresiones

Una expresión puede fallar al compilar el JRXML o al llenar el informe. Un error de sintaxis o una referencia a un campo/variable inexistente suele detectarse durante la compilación. Una operación válida sintácticamente puede fallar en tiempo de llenado, por ejemplo al invocar un método sobre null. La depuración profesional combina cuatro evidencias: Problems, Preview, Console y una ejecución automatizada del runtime.

CICLO DE DEPURACIÓN

1. Guardar JRXML
2. Compilar
 - └ error → Problems
 - └ correcto
3. Preview con datos
 - └ error → Console / stack trace
 - └ correcto
4. Ejecutar GeneradorInformeConcepto
5. Verificar .jasper + JasperPrint + PDF

Qué representa el diagrama: la secuencia usada para separar errores de diseño, compilación y llenado.

Por qué es relevante: evita corregir a ciegas. Cada fase confirma una propiedad distinta del informe.

El checkpoint M2/2.6 se valida además en GitHub Actions. La ejecución real procesa 14 libros, genera un JasperPrint de **3 páginas** y exporta un PDF firmado correctamente. Esa evidencia permite distinguir la simulación didáctica de un resultado realmente ejecutado.

Resumen rápido de la teoría

- Las expresiones JRXML usan sintaxis Java y devuelven un valor.
- \$F{} accede al registro actual, \$P{} a parámetros y \$V{} a variables.
- Los operadores aritméticos, lógicos, de comparación y ternarios pueden combinarse.
- Los métodos Java pueden invocarse desde una expresión, pero deben controlarse los valores nulos.
- Los parámetros conectan la aplicación con el informe.
- Las variables encapsulan cálculos y agregaciones del motor.
- REPORT_COUNT cuenta registros del informe.
- PAGE_COUNT cuenta registros de la página actual; no es el total de páginas.
- El total final de páginas se obtiene en este curso con PAGE_NUMBER y evaluationTime="Report".
- Problems, Preview, Console y la ejecución E2E forman el ciclo de depuración.

Validación técnica final del módulo

Los checkpoints 2.1 a 2.6 se compilan y ejecutan con Temurin JDK 8 y JasperReports Library 6.20.0. La validación genera el .jasper, llena un JasperPrint con la fuente de datos del checkpoint y exporta un PDF real. La matriz M2 - Validacion end-to-end ejecuta los seis checkpoints 2.1–2.6. El identificador

del run de cierre se registra en `VALIDACION_M2.md` después de la ejecución final sobre la revisión documental vigente. En 2.6 la ejecución real informa 14 registros y un JasperPrint de 3 páginas.